PROGRAMACIÓN PARA MÓVILES • PERSISTENCIA EN LA NUBE

Backend en la Nube con Supabase y Flutter

De bases locales en el teléfono a datos compartidos en la nube

Límites locales

- **Un solo equipo:** SQLite y Hive CE guardan datos dentro del dispositivo.
- **Sin sincronía:** Los cambios no se comparten con otros teléfonos.
- **Riesgo local:** Al borrar la app sin copia, los datos se pierden.

Salto a la nube

- **Multi-pantalla:** El usuario accede a sus datos desde cualquier equipo.
- **Tiempo real:** Las actualizaciones llegan al instante a todos.
- **Menos gestión:** Un BaaS ahorra configurar servidores propios.

Por qué Supabase

- **SQL en la nube:** Une la agilidad de Firebase con PostgreSQL real.
- **Código abierto:** Basado en estándares abiertos sin bloqueo.
- **Integración simple:** Conecta con Flutter sin herramientas CLI extras.

02. ¿QUÉ ES SUPABASE?

Supabase brinda un BaaS sobre PostgreSQL

Servicios integrados en la plataforma

- **Base PostgreSQL:** Motor relacional SQL completo con API REST generada al instante.
- **Auth y Storage:** Gestión de usuarios, sesiones seguras y subida de archivos.
- **Realtime y Edge:** Escucha cambios en vivo por WebSockets y ejecuta funciones.



02.1 COMPARATIVA CON LO VISTO EN FIREBASE

Supabase combina la agilidad de Firebase con el poder de SQL

Característica	Firebase (Cloud Firestore)	Supabase (PostgreSQL)
Modelo de datos	NoSQL orientado a colecciones y documentos	Relacional SQL con tablas, filas y columnas
Configuración CLI	Exige npm, firebase-tools y flutterfire configure	Sin CLI externa: usa URL y Anon Key directo
Seguridad de acceso	Reglas propias (modo prueba o producción)	Row Level Security (RLS) estándar de Postgres
Paquete en Flutter	firebase_core + cloud_firestore	supabase_flutter (^2.18.3 en pub.dev)
Consultas en código	db.collection('persona').get() / .add()	supabase.from('persona').select() / .insert()

03. CREAR UN PROYECTO EN SUPABASE.COM

Tres pasos configuran el proyecto y la tabla en la consola web

1. Nuevo Proyecto

- **Registro web:** Ingresa a supabase.com con GitHub o tu correo.
- **Configuración:** Define nombre, clave de base y región.
- **Panel listo:** Se crea tu base PostgreSQL en segundos.

2. Tabla persona

- **Table Editor:** Abre Table Editor y pulsa Create a new table.
- **Columna id:** Queda como int8, Primary Key y autoincremental.
- **Columna nombre:** Añade el campo nombre de tipo text y guarda.

3. Permisos (RLS)

- **Seguridad activa:** Supabase activa Row Level Security por defecto.
- **Modo práctica:** Desmarca Enable RLS para pruebas en clase.
- **En producción:** Configura políticas RLS para controlar accesos.

04. INTEGRACIÓN DE FLUTTER + SUPABASE

Conecta Flutter

Credenciales e instalación del paquete

- **Obtener claves API:** Copia Project URL y anon public key en Project Settings > API.
- **Un solo comando:** Ejecuta flutter pub add supabase_flutter en tu terminal.

```dart
Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();

  await dotenv.load(fileName: '.env');

  await Supabase.initialize(
    url: dotenv.get('SUPABASE_URL'),
    publishableKey:
dotenv.get('SUPABASE_PUBLISHABLE_KEY'),
  );

  runApp(const MyApp());
}
```

04.1 CONFIGURACIÓN EN MAIN.DART

Supabase.initialize prepara la conexión

Arranque asíncrono en tres pasos

- **Asegurar binding:** Llama a WidgetsFlutterBinding.ensureInitialized() en main().
- **Inicializar cliente:** Ejecuta await Supabase.initialize(url: '...', anonKey: '...').
- **Acceso global:** Obtén el cliente donde lo necesites con Supabase.instance.client.

05. EJEMPLO DE USO: ARCHIVO SUPABASE_SERVICES.DART

Cuatro funciones asíncronas resuelven persona

Consulta e Inserción

- **Cliente activo:** final supabase = Supabase.instance.client; reemplaza a Firestore.
- **getPersona():** await supabase.from('persona').select('id, nombre') lee las filas.
- **addPersona(nombre):** await supabase.from('persona').insert({'nombre': nombre}) agrega datos.

Modificación y Baja (UPDATE / DELETE)

- **updatePersona(id, nom):** await supabase.from('persona').update({'nombre': nom}).eq('id', id).
- **borrarPersona(id):** await supabase.from('persona').delete().eq('id', id) borra la fila.
- **Filtro relacional:** El método .eq('id', id) equivale a la cláusula SQL WHERE id = X.

05 TIEMPO REAL Y ACTIVIDAD PRÁCTICA

StreamBuilder escucha cambios en vivo

Sincronización en vivo y desafío en clase

- **Streams en Supabase:** supabase.from('persona').stream(primaryKey: ['id']) emite en vivo.
- **Actualización sola:** Con StreamBuilder los cambios de otros usuarios aparecen al instante.
- **Práctica de hoy:** Crear la tabla persona en supabase.com y probar el ABM completo.